pid 的一个典型值是：pid=('storage', LazyStorageKind(data_type='F16'), '0', 'cpu', 131072000)。storage='storage' 是固定字符串；filename_stem='0' 用于拼接成权重文件，0 是指模型第 0 个部分（embed_tokens.weight）；131072000 是该部分的参数量，对应 32 000 × 4 096。

find_class 用来建立模型中不同类别与相应处理函数的对应关系。比如 ('torch._utils', '_rebuild_tensor_v2') 对应的是 lazy_rebuild_tensor_v2 函数。对应不同 pid，persistent_load 调用不同的处理函数。

程序运行目的是从模型权重文件中读得类似以下格式字典数据（仅为第 0 部分）：

'model.embed_tokens.weight': LazyTensor(shape=[32000, 4096], data_type='F16', description='pickled storage_offset=0 in storage data_type=F16 path-in-zip=pytorch_model-00001-of-00002/data/0 path=C:\\llama2\\llama-2-7b-hf\\pytorch_model-00001-of-00002.bin')

这些数据是先从 persistent_load 函数中根据 pid 生成的 storage data_type=F16 path-in-zip=pytorch_model-00001-of-00002/data/0 path=C:\\llama2\\llama-2-7b-hf\\pytorch_model-00001-of-00002.bin，再由 lazy_rebuild_tensor_v2 合并上 shape=[32000, 4096], data_type='F16', description='pickled storage_offset=0 数据。

源程序中首先导入所需的库和模块，定义变量和三个类：LazyStorageKind、LazyStorage、LazyTensor。

```
import zipfile
import pickle
from pathlib import Path
from typing import (IO, Any, Callable, Optional, List, Dict)
from dataclasses import dataclass
import numpy as np

DT_F16 = 'F16'
DT_F32 = 'F32'
DT_BF16 = 'DT_BF16'
DT_I32 = 'DT_I32'

DataType = 'F16'

@dataclass
class LazyStorageKind:
    data_type: DataType

@dataclass
class LazyStorage:
    kind: LazyStorageKind
    description: str

@dataclass
class LazyTensor:
    shape: List[int]
    data_type: DataType
    description: str
```

定制化专用的反序列化类 LazyUnpickler，重写 pickle.Unpickler 类的 persistent_load、find_class 函数。

```
class LazyUnpickler(pickle.Unpickler):
    def __init__(self, fp: IO[bytes], data_base_path: str, zip_file: zipfile.ZipFile):
        super().__init__(fp)
        self.data_base_path = data_base_path
```



```
        self.zip_file = zip_file

    def persistent_load(self, pid: Any) -> Any:
        assert pid[0] == 'storage'
        assert isinstance(pid[1], LazyStorageKind)
        data_type = pid[1].data_type
        filename_stem = pid[2]
        filename = self.data_base_path + '/' + filename_stem
        info = self.zip_file.getinfo(filename)

        description = f'storage data_type={data_type} path-in-zip={filename} path={self.
zip_file.filename}'
        return LazyStorage(kind=pid[1], description=description)

    # @staticmethod
    def lazy_rebuild_tensor_v2(storage: Any, storage_offset: Any, size: Any, stride: Any,
                               # pyright: ignore[reportSelfClsParameterName]
                               requires_grad: Any, backward_hooks: Any, metadata: Any = None)
-> LazyTensor:
        assert isinstance(storage, LazyStorage)

        description = f'pickled storage_offset={storage_offset} in {storage.description}'
        return LazyTensor(list(size), storage.kind.data_type, description)

    CLASSES: Dict[Any, Any] = {
        ('torch._utils', '_rebuild_tensor_v2'): lazy_rebuild_tensor_v2,
        ('torch', 'BFloat16Storage'): LazyStorageKind(DT_BF16),
        ('torch', 'HalfStorage'): LazyStorageKind(DT_F16),
        ('torch', 'FloatStorage'): LazyStorageKind(DT_F32),
        ('torch', 'IntStorage'): LazyStorageKind(DT_I32)
    }

    def find_class(self, module: str, name: str) -> Any:
        if not module.startswith('torch'):
            return super().find_class(module, name)
        return self.CLASSES[(module, name)]
```

主函数，打开指定的一个 hf 格式的模型权重文件 path，用 zipfile 将其解压，找到压缩文件中的 data.pkl 文件，用 LazyUnpickler 将其反序列化，得到模型字典数据 as_dict，可以循环显示模型每个部分的参数，累计得到总参数量和字节数。

```
if __name__ == '__main__':
    path = Path("C:\\llama2\\llama-2-7b-hf\\pytorch_model-00001-of-00002.bin")

    fp = open(path, 'rb')

    zf = zipfile.ZipFile(fp)

    pickle_paths = [name for name in zf.namelist() if name.endswith('.pkl')]
    assert len(pickle_paths) == 1, pickle_paths

    pickle_fp = zf.open(pickle_paths[0], 'r')
    unpickler = LazyUnpickler(pickle_fp,
                              data_base_path=pickle_paths[0][:-4],
                              zip_file=zf)

    model = unpickler.load()
    as_dict = dict(model.items())

    nParams = 0
    nBytes = 0
```